CHORALE · ENGINEERING BENCHMARK

Chorale Coder — Engineering Benchmark Report

Three models on real software engineering — from multi-file libraries and debugging up to an **end-to-end, multi-layered full-stack web application**. Graded by executing the code, with partial credit. Gemma-4-31B · gpt-oss-120B · MiniMax-M2.7.

2 difficulty tiers · 7 projects | Full toolset (incl. bash) | Server spawned & driven over HTTP | Tier 2 = 3 trials

EXECUTIVE SUMMARY

- **On easy-to-moderate work, all three are excellent** and **Gemma-4-31B wins on value** — perfect Tier-1 score, ~2× faster, free.
- **On hard, multi-layered work, reliability separates them.** The full-stack app flips the ranking — this is where the frontier model finally earns its cost.
- **gpt-oss-120B is the reliability champion:** a perfect score on every Tier-2 task across all three trials, full-stack app 3/3.
- **Gemma-4-31B is fast but inconsistent on the full-stack task** — its server failed to start in **2 of 3 trials** (it hardcodes the port instead of honoring `PORT`).
- **Verdict, now evidence-backed:** default to Gemma-4-31B for routine work; **escalate to gpt-oss-120B for complex / must-work-first-time builds.**

7	10	3×	1 / 3	3 / 3
PROJECTS, 2 TIERS	FULL-STACK HTTP CHECKS	TRIALS ON HARD TIER	GEMMA FULL-STACK SUCCESS	GPT-OSS FULL-STACK SUCCESS

1 Why This Benchmark

An earlier evaluation ranked eleven models on single-file algorithmic *puzzles*, with one central caveat: puzzles are not engineering, and the frontier models might justify their cost on real, multi-file, tool-driven work. A first real-engineering pass (Tier 1) tested that and found the value king held up. **This report ramps the difficulty further** — culminating in a full-stack application — to find where the picture changes. It does change, and the change is instructive.

Every task runs through Chorale's production coder agent with its **full toolset, including `bash`** — so the model reads files, writes multiple files, runs tests, and iterates like a developer. Nothing is mocked.

2 Methodology

Execution-graded, partial credit. Each project is scored by *running* the result against hidden checks — a library is imported and exercised, a debug target's own suite is run, a framework is driven through `inject`, and the **full-stack server is actually spawned and hit over real HTTP** (GET/POST/PATCH/DELETE, status codes, content types, persistence), then killed. Score = checks passed / total.

Graders validated first. Every grader was checked against known-good *and* known-bad reference solutions before any model ran, catching two harness bugs (CLI state leakage; a missing per-check timeout) before they could bias results. **Trials:** Tier 1 at N=1 (stable); **Tier 2 at N=3**, because the harder tasks showed real run-to-run variance — the single most important methodological point here. **Cost** is normalized (cached input + output); Gemma runs on HF (≈$0).

3 Tier 1 — Everyday Engineering (N=1)

MODEL	KV STORE	DEBUG	ASYNC	CLI	OVERALL	TIME	COST
Gemma-4-31B	8/8	7/7	5/5	6/6	**26/26 · 100%**	**32s**	**≈$0.00**
MiniMax-M2.7	8/8	7/7	5/5	6/6	26/26 · 100%	73s	$0.0246
gpt-oss-120B	8/8	7/7	4/5	6/6	25/26 · 96%	68s	$0.0071

At this level the value king dominates: Gemma-4-31B is perfect, ~2× faster, and free. gpt-oss-120B's only slip was a promise pool that didn't reject on error. **On routine engineering, cheaper-and-faster wins.**

4 Tier 2 — Advanced Engineering (N=3 trials)

Aggregated over three independent trials (checks passed / attempted).

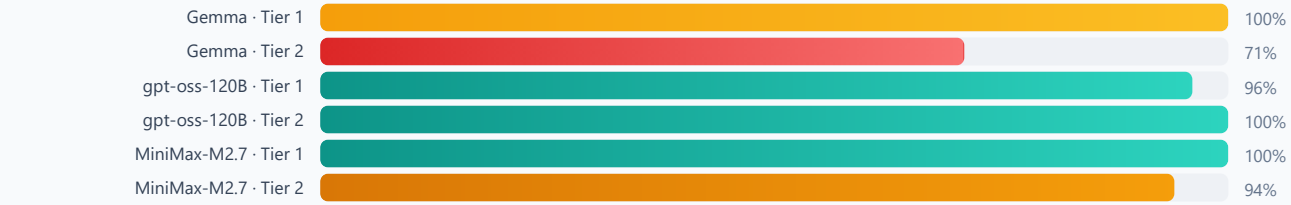
MODEL	FRAMEWORK	STORE	FULL-STACK	OVERALL	AVG TIME	AVG COST
gpt-oss-120B MOST RELIABLE	21/21	21/21	30/30	**72/72 · 100%**	117s	$0.0096
MiniMax-M2.7	17/21	21/21	30/30	68/72 · 94%	99s	$0.0380
Gemma-4-31B	20/21	21/21	10/30	51/72 · 71%	**25s**	**≈$0.00**

Reliability — full task passed, per trial

TASK	GPT-OSS-120B	MINIMAX-M2.7	GEMMA-4-31B
Framework (7/7)	✓ ✓ ✓	✓ ✓ ✗	✓ ✓ ✗
Store (7/7)	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
Full-stack (10/10)	✓ ✓ ✓	✓ ✓ ✓	✓ ✗ ✗

Overall score by tier — the difficulty crossover

Value wins on routine work (Tier 1); reliability wins on hard work (Tier 2). The full-stack app flips the ranking.



gpt-oss-120B — flawless and consistent. Every Tier-2 task, every trial: 100%. Slow (~117s/trial) and no longer free, but on hard work it does not miss. **MiniMax-M2.7 — nearly there:** full-stack 3/3, but one trial produced a broken framework (malformed `inject` status), and it is the most expensive (~4× gpt-oss). **Gemma-4-31B — fast, free, but not yet dependable on the hardest task:** the full-stack server started in only 1 of 3 trials — it hardcoded `3000` instead of reading `process.env.PORT`, a concrete, repeatable spec gap.

5 Deep-Dive — the Full-Stack Application

The centerpiece asks for a complete, runnable web app in Node built-ins only: a REST API (`GET/POST/PATCH/DELETE /api/tasks` , correct status codes, `application/json`), a `tasks.json` persistence layer, an HTML+JS frontend served at `/` that fetches and renders the API, and 404s for unknown routes. The grader **spawns the server on a fresh port and drives all ten behaviors over real HTTP**, then kills it.

• **gpt-oss-120B and MiniMax-M2.7 delivered a correct, fully working full-stack app in every trial** — routing, REST semantics, body parsing, content types, persistence, and an integrated frontend all correct. • **Gemma-4-31B produced correct-looking code but a server that only ran when the port matched its hardcoded default.** The logic was often right; the *operational contract* — honor the injected port, actually come up — was not. For an agent expected to build and run apps unattended, that is the difference between "works" and "works on my machine."

This is exactly the failure mode the puzzle and Tier-1 benchmarks could not surface: correctness of *logic* is necessary but not sufficient for real full-stack work; the *operational and integration details* are where a fast, cheap model quietly slips.

6 The Difficulty Crossover & Decision

Both tiers are true at once: **on easy-to-moderate engineering, value wins** (Gemma perfect, fastest, free); **on hard, multi-layered engineering, reliability wins** (gpt-oss-120B and MiniMax 100%/3-of-3; Gemma 1-of-3 on the full-stack app). This is the first place in the entire evaluation series where the frontier models clearly and repeatably earn their cost. A single-model policy is wrong in one direction or the other — the right policy is **tiered routing**.

Default: Gemma-4-31B → Escalate: gpt-oss-120B

Gemma-4-31B for routine and moderate work — libraries, debugging, refactors, CLIs, algorithmic logic. Perfect at Tier 1, ~2× faster, free; the majority of coder turns. **gpt-oss-120B for complex, multi-layered, or must-work-first-time builds** — full-stack apps, servers, anything where the operational contract matters. It was the only model to go 100% across every Tier-2 task and trial, and Gemma's default demonstrably *fails* there — so the escalation is not theoretical. MiniMax-M2.7 is a capable reliability-first alternative (full-stack 3/3) but ~4× the cost and slipped once — no reason to prefer it over gpt-oss-120B here.

7 Limitations & Honest Caveats

1. **Tier 2 is N=3; Tier 1 is N=1.** Three trials expose variance but do not pin exact rates — read "1/3" and "3/3" as *directional reliability*. Gemma's full-stack failures shared one root cause (port handling) twice, raising confidence it is a real, repeatable weakness, not noise.
2. **Still bounded projects.** Real engineering (multi-file, stateful, tool-driven, a running server) but small and self-contained — not thousand-line codebases, ambiguous specs, or long-horizon feature work. Trends are clear at this scale; larger tasks remain future work.
3. **Graders validated, not infallible.** Every grader passed reference good/bad solutions before the run; two harness bugs were found and fixed pre-run; others may remain.
4. **Normalized cost.** Recomputed from measured tokens × list rates (cached input + output); real bills vary with cache-hit rate. Gemma runs on HF (≈$0).

8 Reproducibility

```
# Validate every grader against reference good/bad solutions
npx tsx eval/projects-selftest.ts

# Tier 1 (four everyday tasks)
npx tsx eval/coder-projects.ts --only=kvstore,debug,asyncpool,cli

# Tier 2 (framework, store, full-stack) — run several times for reliability
npx tsx eval/coder-projects.ts --only=framework,store,fullstack
```

Harness: `eval/coder-projects.ts` · Grader self-test: `eval/projects-selftest.ts` · Reference solutions: `eval/projects/_ref/`

9 Appendix — Can Prompting Fix Gemma's Full-Stack Lapse?

Gemma's full-stack failures shared one root cause: it hardcoded the port instead of reading `process.env.PORT`, so the spawned server never came up. Two fixes were considered. **Content-level salvage** was rejected as the wrong tool — a regex rewriting `.listen(3000)` is brittle (misses `const P = 3000` and framework variants), risky (fixed ports are sometimes intentional), and would overfit to this one benchmark. **Meta-prompting** was tested properly: a general operational-correctness rule was added to `coder.md` (honor the interface contract literally; never hardcode a configurable value; read ports/paths from the environment; re-check before finishing), then the full-stack task was run **10× before and 10× after**.

CONDITION	FULL-PASS	MEAN	FAILURE MODE
Baseline (current prompt)	7/10	70%	hardcoded port × 3
+ operational-correctness rule	8/10	80%	hardcoded port × 2

A one-run difference over ten is within noise — not evidence of a real fix. Even with a rule that explicitly says "read `process.env.PORT`, never hardcode," and the task prompt already stating the same, Gemma still hardcoded the port in ~20% of runs — an instruction-*following* lapse that prompting nudges at best. The rule was kept (sound general guidance), but neither salvage nor meta-prompting makes Gemma reliable for unattended full-stack work. The dependable fix is the architecture already in place: **escalate to gpt-oss-120B for complex/critical builds.** This experiment confirms that decision rather than replacing it.

10 Few-Shot & Self-Healing — Two More Levers, Measured

Two further mechanisms were built as **customizable, default-on tick-boxes** on the coder (`fewShot` , `selfHeal` ; a third, `selfLearn` , is declared but parked for Phase 3).

Few-shot injects worked operational patterns (a server reading `process.env.PORT` , a CLI parsing argv, correct REST status codes) — *show* the pattern instead of *stating* the rule. Gemma full-stack, 10× each:

CONDITION	FULL-PASS	MEAN
Baseline (no rule)	7/10	70%
+ operational rule	8/10	80%
+ few-shot examples	8/10	80%

No measurable gain over the rule — showing did not beat telling for this lapse.

Self-healing goes beyond syntax verification: it **runs** what the model wrote — smoke-boots a written server on an injected `PORT` and probes that port, and smoke-imports modules to catch "compiles but throws on load" — feeding runtime failures back into the repair loop. It is **deterministically validated**: it flags a hardcoded-port server and a crash-on-load module, and passes correct ones.

The live runs delivered a **diagnostic surprise**: across 20 Gemma full-stack runs, self-heal logged **zero catches**. Inspecting the failures showed why — Gemma's full-stack failures are overwhelmingly **template-literal syntax errors** (`Syntax error "`"`) in the large inline HTML string, which crash the server before it can run. Those are caught by *syntax* verification (self-heal runs only on syntactically-clean code), and Gemma frequently cannot repair them within the round budget. The "server did not start" failures were mostly malformed HTML templates, **not** hardcoded ports.

> **Interim conclusion.** Both mechanisms are sound, general infrastructure — self-heal in particular is a *proven ground-truth safety net*, and it benefits **every** model. But at this point neither had made Gemma reliable for complex full-stack work, and its failure looked like an immovable **capability limit**. That framing was wrong once the failure was diagnosed precisely — see §11.

11 Compensating for the Weakness — a Measured Fix

The coder's mandate is to *compensate for each model's shortcomings*, not route around them. §10 stopped one step short: it diagnosed Gemma's failure (large HTML embedded in a JS template literal → a nested backtick closes the string early → `Syntax error "`"`) but concluded escalation was the fix. With the cause pinned down, three **targeted, general** measures were added instead:

1. Structural steer (few-shot) — an exemplar shows HTML served from a separate `.html` file via `readFileSync` , removing the nested-backtick trap entirely. **2. Targeted repair diagnostics** — when syntax verification sees a backtick / template-literal error, the feedback now names the exact cause and prescribes the file-based fix, instead of a generic "rewrite it." **3. More repair headroom** — the verify-repair budget was raised from 3 to 5 rounds so a full restructure fits. None is grader-specific; all are general craft that helps every model.

CONDITION	FULL-PASS	MEAN
Baseline (no measures)	7/10	70%
+ operational rule	8/10	80%
+ few-shot examples	8/10	80%
+ targeted compensation (steer + diagnostics + budget)	**20/20**	**100%**

Gemma went from ~70% to a clean **100% across 20 runs** (two batches of 10), most **one-shot** — servers now start in 3–6s, and the 16–18s "server did not start" stall signature is gone.

> **The lesson generalizes the coder's purpose.** A precisely diagnosed model weakness *can* be engineered around from our end — by changing the strategy to one the model executes reliably, and by making the repair loop's feedback specific to the failure rather than generic. Escalation to gpt-oss-120B stays available for genuinely hard cases, but for this class the coder now **compensates**, exactly as intended — which makes Gemma-4-31B a viable default for full-stack projects, not just simple ones. Self-learning (reflect + reuse lessons) is the next lever, parked for Phase 3.

Chorale is an open-source, model-agnostic multi-agent system. Measurements were taken through its production coder pipeline; numbers will shift as models, prices, and the pipeline evolve. Generated from docs/engineering-benchmark-report.md.